Atlas Intelligence

Can show publicly	Partial
Category	Finance & Market Intelligence or Business Intelligence & Analytics
Country	France, Morocco, Qatar, Saudi Arabia, Spain, UAE, USA
Industry	Market intelligence, enterprise intelligence, finance, operations risk
Main features	- - -
Project link	https://atlas-intelligence.vercel.app/scan
Project type	Internal Product / Demo
Proof available	Public link: Screenshots: Demo: Testimonials: Metrics:
Status	Delivered
Technical stack	- - -

I'll audit this repository systematically to produce a factual portfolio report. Let me start by exploring the project structure and key files.Let me explore the directory structure and examine more files.# Project Repository Report

1. Project identity

- **Project name:** Intelligence Shell Starter (starter-intelligence-shell)
- **Repository name:** atlas-intelligence-shell
- **Project category:** UI Framework / Design System Starter
- **Apparent project type:** Internal product / Prototype / Reusable starter template
- **Public-facing or internal-facing:** Internal-facing (designed for internal team use and AI agent extension)
- **Main user/customer:** Internal Atlas team; AI agents (Cursor, Claude) building new intelligence products

2. One-sentence factual description

A copy-ready Next.js starter template that extracts the canonical Atlas Intelligence terminal shell with four operational modes (Scan, Investigate, Decide, Execute) and a vendored design system, intended to be cloned and customized with product-specific demo data and content.

3. What the project does

- **Provides a runnable shell architecture** — Topbar, left sidebar, mode switcher, resizable multi-panel workbench, right context panel, bottom status strip with keyboard shortcuts and global state management via Zustand.
- **Implements four operational modes** — Scan (watch market and asset data), Investigate (entity graphs and timeline metrics), Decide (scenario builder with constraints and recommendations), Execute (active workflows and task management).
- **Bundles demo content** — Includes placeholder data (market indices, assets, evidence items, entities, workflows, tasks, decisions) in `lib/demo-data/` intended to be replaced by product-specific content.
- **Encodes UI DNA and design discipline** — Terminal-like aesthetic with compact density, border-led panel separation, mono numeric treatment, and restrained accent usage via Tailwind CSS v4 tokens.

- **Includes a nested design system** — Vendored `atlas-design-system/` containing reusable primitives, charts, tokens, and layouts for shared component development.
- **Provides AI-agent-ready prompting framework** — `prompt/` folder with Architect/Builder/Critic agents, spec templates, prompt compiler, and checklists to turn product ideas into prototypes using LLMs.

4. Confirmed implemented features

Feature	What it does	Evidence file paths	Confidence
Scan Mode	Displays market watch indices, tracked assets, real-time charts, operational alerts, geo-pulse risk data, and evidence stream filtered by source type	`components/modes/scan-mode.tsx`, `lib/demo-data/scan.ts`	High
Investigate Mode	Shows entity relationship graph with interactive nodes, timeline series with price/events, metrics rows with trends, and evidence vault searchable by provider	`components/modes/investigate-mode.tsx`, `lib/demo-data/investigate.ts`	High
Decide Mode	Scenario builder with decision selector, constraint sliders (budget, risk, time horizon), option comparison chart, recommendations prioritized by confidence, and assumptions/caveats	`components/modes/decide-mode.tsx`, `lib/demo-data/decide.ts`	High
Execute Mode	Active workflows list with progress bars, task details per workflow with priority/status/assignee, playbook reference	`components/modes/execute-mode.tsx`, `lib/demo-data/execute.ts`	High

Feature	What it does	Evidence file paths	Confidence
	library, and audit log of actions		
Resizable panel layout	Multi-panel workbench with horizontal and vertical ResizablePanelGroup; draggable handles; collapsible panels via PanelFrame	`components/shell/panel-frame.tsx`, `components/ui/resizable.tsx`, modes components	High
Left sidebar navigation	Collapsible sidebar; "Watch" and "Feeds" tabs; entity list grouped by kind (assets, companies, facilities, regions); each entity shows sparkline trend and value delta	`components/shell/sidebar.tsx`, `lib/demo-data/shell.ts`	High
Right context panel	Toggles open/closed; displays selected entity details and related evidence items with source type, provider, reliability score, and confidence bar	`components/shell/right-context-panel.tsx`	High
Bottom status strip	Shows activity indicator (LIVE), database latency (31ms mock), last ingest timestamp (2s ago mock), and current selection breadcrumb (entity kind > subtitle)	`components/shell/bottom-strip.tsx`	High
Mode switcher	Horizontal button row (SCAN, INVESTIGATE, DECIDE, EXECUTE) with keyboard shortcut hints (1–4); Ctrl+K or Cmd+K opens command palette	`components/shell/mode-switcher.tsx`, `components/shell/command-palette.tsx`	High

Feature	What it does	Evidence file paths	Confidence
Keyboard shortcuts	Keys 1/2/3/4 navigate modes; "/" opens command bar for entity search; Escape clears selection; Ctrl+K opens command palette	`components/shell/app-shell.tsx`, `components/shell/command-bar.tsx`	High
Command palette	Searchable global navigation; filters modes, entities, evidence, and command history; supports fuzzy matching	`components/shell/command-palette.tsx`	High
Global state (Zustand store)	Persists to localStorage: active mode, selection (entityId/evidenceId), left/right collapse state, rail section, selected symbol, command history	`lib/store/shell-store.ts`	High
Charts & visualization	Area charts (market scan), line charts (timeline investigate), bar charts (decide risk), sparklines (sidebar trends) via Recharts	`components/modes/scan-mode.tsx`, `components/modes/investigate-mode.tsx`, `lib/demo-data/scan.ts`	High
Design tokens & styling	Tailwind CSS v4 with custom `terminal-*` tokens (base, nested, panel, accent) and typography utilities (`terminal-label`); dark terminal aesthetic	`app/globals.css` (referenced), `package.json`	High
TypeScript strict mode	Full TypeScript coverage with types for Mode, RailSection, Selection, ShellEntity, EvidenceItem, ModeDefinition	`lib/types.ts`, `tsconfig.json`	High

5. Partially implemented or unclear features

Feature / claim	Why it is unclear or partial	Evidence file paths	What needs confirmation
Real data ingestion	All data is hardcoded demo/mock data in `lib/demo-data/`. No API calls, database connections, or real data sources are present.	`lib/demo-data/*.ts`	Whether this is intended as template-only or if live data integrations are expected in derived products
Entity relationship logic	Graph nodes and edges are static mock data with no dynamic relationship calculation or real network analysis.	`lib/demo-data/investigate.ts`	Whether entity relationships are meant to be computed from real data sources
Workflow execution and task updates	Tasks and workflows are static. No UI actions (e.g., completing a task, updating progress) persist state beyond Zustand in-memory store.	`components/modes/execute-mode.tsx`	Whether workflow state should sync to a backend or remain client-side demo
Evidence reliability scoring	Evidence items have a `reliability` field (0–100) but scoring logic is not visible; no algorithm for how reliability is calculated.	`lib/demo-data/shell.ts`	How reliability is computed in a real product
Confidence metrics	Many entities show confidence scores (84, 73, 57) but no visible logic for confidence calculation or validation.	`lib/demo-data/decide.ts`, `lib/demo-data/investigate.ts`	Source and calculation method for confidence values

Feature / claim	Why it is unclear or partial	Evidence file paths	What needs confirmation
Latency and ingest metrics	Bottom strip shows hardcoded "Latency: 31ms" and "Last ingest: 2s ago" — not reflecting real system state.	`components/shell/bottom-strip.tsx`	Whether these should connect to real backend metrics
AI prompt system readiness	The `prompt/` folder contains agent definitions (Architect, Builder, Critic) and spec templates, but is documented as operational framework, not yet deployed at scale.	`prompt/README.md`, `prompt/HOW_TO_USE.md`, `prompt/agents/`	Whether the agent pipeline is production-tested or still experimental

6. Mocked, hardcoded, or demo-only parts

Area	What appears mocked/demo-only	Evidence file paths	Risk if claimed publicly
All data in Scan mode	Market indices, tracked assets, chart series, market pulse, ops alerts, geo-pulse, evidence stream are all static objects.	`lib/demo-data/scan.ts`	High: Would imply live market data; confuse users expecting real indices
All data in Investigate mode	Graph nodes/edges, timeline series, metrics rows, evidence vault are hardcoded. No timeline is real.	`lib/demo-data/investigate.ts`	High: Would imply real entity network and historical analysis
All data in Decide mode	Decisions, scenarios, options,	`lib/demo-data/decide.ts`	High: Would imply analytical

Area	What appears mocked/demo-only	Evidence file paths	Risk if claimed publicly
	recommendations, assumptions are static. No scenario modeling engine.		recommendation system; no optimization backend
All data in Execute mode	Workflows, tasks, playbooks, audit log are hardcoded. Clicking buttons does not create real tasks or log real actions.	`lib/demo-data/execute.ts`	High: Would imply operational control system; users expect state persistence
Bottom strip metrics	Latency "31ms", ingest "2s ago" are strings, not real measurements.	`components/shell/bottom-strip.tsx`	Medium: Could mislead about system health if not replaced
Entity graph visualization	SVG graph is static coordinate-based, not a dynamic force graph or real relationship engine.	`components/modes/investigate-mode.tsx`	Medium: Would imply live network analysis
Command history and search	Command bar filters against static `shellEntities`; no real search backend or API.	`components/shell/command-bar.tsx`, `components/shell/command-palette.tsx`	Medium: Limited to demo entities unless replaced
Status indicators	"LIVE" status, progress bars (62%, 44%, 18%), task counts are mock values.	`components/shell/bottom-strip.tsx`, `lib/demo-data/execute.ts`	Medium: Would imply real operational status

7. Technical stack

Layer	Technologies confirmed	Evidence file paths
Frontend framework	Next.js 16.1.1 (App Router); React 19.2.0	`package.json`

Layer	Technologies confirmed	Evidence file paths
Language	TypeScript 5.x; strict mode	`tsconfig.json`, `package.json`
Styling	Tailwind CSS v4.1.9; PostCSS 8.5	`package.json`, `postcss.config.mjs`
UI component library	Radix UI (collapsible, dialog, slider, dismissable-layer, slot)	`package.json`
Resizable panels	react-resizable-panels 2.1.7	`package.json`, `components/ui/resizable.tsx`
State management	Zustand 5.0.9 (with persist middleware, localStorage)	`package.json`, `lib/store/shell-store.ts`
Command palette / search	cmdk 1.0.4	`package.json`
Charting	Recharts 2.15.4 (Area, Line, Bar charts)	`package.json`, mode components
Virtual scrolling	@tanstack/react-virtual 3.13.16	`package.json`
Icons	Lucide React 0.454.0	`package.json`, all components
Utilities	clsx 2.1.1; class-variance-authority 0.7.1; tailwind-merge 3.5.0	`package.json`
Animations	tw-animate-css 1.3.3	`package.json`
Package manager	pnpm 9.15.0	`package.json`
Build tool	Next.js Turbopack (via `pnpm dev --turbopack`)	`package.json` scripts
Type definitions	next-env.d.ts auto-generated	`next-env.d.ts`

Layer	Technologies confirmed	Evidence file paths
Database	Not found in repo.	
Authentication	Not found in repo.	
API/Backend	Not found in repo. (Starter is frontend-only.)	
File storage	Not found in repo.	
AI/LLM integration	Not in runtime code; documented AI agent framework for *generating* prototypes using external LLMs (Cursor, Claude).	`prompt/` folder, `docs/ai-agent-guide.md`
Payments/e-commerce	Not found in repo.	
CMS	Not found in repo.	
Analytics	Not found in repo.	
Deployment	Not found in repo. (No Docker, k8s, serverless config.)	
Testing	Structure test file referenced in docs; no test framework in dependencies.	`prompt/checklists/pre_generation_checklist.md`, `docs/ai-agent-guide.md`
Nested design system	Vendored `atlas-design-system/` with primitives, charts, tokens, hooks, layouts, patterns, intelligence components	`atlas-design-system/README.md`

8. Routes, pages, and user flows

Route/Page	Purpose	Evidence file paths
`/`	Defaults to `/scan`	`components/shell/app-shell.tsx` (route logic)
`/scan`	**Scan Mode** — Monitor market watch, track assets, view operational alerts, browse evidence stream by source type	`components/modes/scan-mode.tsx`, `lib/demo-data/scan.ts`
`/investigate`	**Investigate Mode** — Explore entity relationship graph, view timeline of price/events, inspect metrics with trends, search evidence vault	`components/modes/investigate-mode.tsx`, `lib/demo-data/investigate.ts`
`/decide`	**Decide Mode** — Build scenarios with constraints (budget, risk, time horizon), compare decision options, review recommendations and assumptions	`components/modes/decide-mode.tsx`, `lib/demo-data/decide.ts`
`/execute`	**Execute Mode** — Manage active workflows with progress tracking, view tasks with priority/assignee/status, reference playbooks, review audit log	`components/modes/execute-mode.tsx`, `lib/demo-data/execute.ts`

Global flows:

- **Mode navigation:** Keys 1/2/3/4 jump between modes; buttons in ModeSwitcher; command palette with Ctrl+K.
- **Entity selection:** Left sidebar or command bar (/) for searching entities; selected entity highlighted; context panel updates on right; bottom strip shows breadcrumb.
- **Evidence linking:** Right context panel filters evidence by selected entity or evidence ID; clicking evidence item marks it selected.
- **Panel resizing:** Drag handles between panels; state persists to Zustand store.
- **Sidebar collapse:** Left and right panels toggle open/closed via collapse buttons; state persists.

9. Data model and backend logic

Entities and data structures:

```
ShellEntity {
  id, label, subtitle, kind (asset | company | facility | reg
ion)
  value, delta, trend[], relatedEvidenceIds[]
}

EvidenceItem {
  id, title, summary, provider, sourceType (news | filing | b
rief | workflow)
  timestamp, reliability (0–100), tags[], relatedIds[]
}

Mode = "scan" | "investigate" | "decide" | "execute"
Selection { entityId?, evidenceId?, decisionId?, taskId?, wor
kflowId? }
```

Demo data structure:

- **Scan:** Market indices, tracked assets, chart series, market pulse (risk index, sectors), ops alerts (HIGH/MEDIUM/LOW), geo-pulse (region risk), evidence stream.
- **Investigate:** Graph nodes (company, facility, region types), edges (supplies, routes, exposed), timeline series (date, price, opsEvents, news), metrics rows (market cap, reliability, variance, inventory, exposure), evidence vault.
- **Decide:** Decisions (category-based), scenarios (with confidence), options (margin impact, delivery delta, risk score, uncertainty bounds), recommendations (priority, rationale, upside/downside), assumptions.
- **Execute:** Workflows (status, progress %), tasks (workflowId, priority, status, assignee, due), playbooks (name, description, steps), audit log (actor, actionType, timestamp).

State management:

- Zustand store persists to localStorage: `activeMode`, `selection`, `leftCollapsed`, `rightCollapsed`, `railSection`, `selectedSymbol`, `commandHistory`.
- No backend API calls; all state is in-memory and in browser storage.

Business logic:

- No real business logic present. Demo content is entirely static.
- Selection state filters what appears in context panels and highlights in UI.
- Constraint sliders in Decide mode are interactive but do not drive any real calculations.

10. AI-specific analysis, if applicable

AI involvement in this project:

1. **Not a runtime AI product.** The shell itself does not call any LLM API or perform inference.
2. **AI-as-meta-tool for *generating* products from this starter.** The `prompt/` folder is an **AI agent prompting framework** designed for external agents (Cursor, Claude) to turn product ideas into prototype implementations *using* this starter.
3. **Agent pipeline:**
   - **Architect Agent** (`prompt/agents/architect_agent.md`) — Validates idea specs and product specs; refuses generation if specs are incomplete.
   - **Builder Agent** (`prompt/agents/builder_agent.md`) — Generates code for a new product by following a compiled system prompt, UI rules, and engineering constraints.
   - **Critic Agent** (`prompt/agents/critic_agent.md`) — Adversarially reviews the generated prototype; outputs FAILURES, MISSING FEATURES, SPEC MISMATCHES, RECOMMENDED FIXES.
4. **Spec-driven code generation workflow** (`prompt/HOW_TO_USE.md`):
   - Fill `idea_spec.md` (problem, target users, core insight).
   - Run Architect → get APPROVED or REFUSED.
   - Expand to full specs: `product_spec.md`, `ui_spec.md`, `data_spec.md`, `behavior_spec.md`, `success_criteria.md`.
   - Run Architect on all six specs.
   - Run pre-generation checklist.
   - Compile system prompt from specs and templates.
   - Send compiled prompt + rules to Builder Agent (Cursor).
   - Run post-generation audit.

- Run Critic Agent for review and recommendations.
- Refine and re-run until Critic approves.

5. **Prompts and templates present:**
   - System prompt template in `prompt/templates/system_prompt_template.md`.
   - UI rules in `prompt/rules/ui_rules.md`, UI DNA in `prompt/rules/ui_dna.md`, engineering rules in `prompt/rules/engineering_rules.md`.
   - Pre-generation and post-generation checklists in `prompt/checklists/`.
   - Compiler logic in `prompt/compiler/prompt_compiler.md`.
6. **Outputs are code, not saved inferences.** The Builder Agent outputs TypeScript/React code directly into the repo; no separate inference log or model outputs are stored.
7. **Evidence of real agent workflow:**
   - The entire `prompt/` folder structure is defined and documented.
   - Agent roles and responsibilities are explicit.
   - The workflow includes adversarial review (Critic) — unusual in typical AI prompting and suggests real operational use.
   - Recommended reading order and phase-by-phase prompts are provided for users.

Conclusion: This is not an AI product for end users; it is an **AI-driven product development framework** intended for internal teams and agents to generate new intelligence dashboards quickly.

11. Portfolio table improvements

Field	Recommended text
Status	Prototype / Internal template (not production)
Industry	Intelligence, analytics, operational dashboards (domain-agnostic framework)
Category	UI framework / Design system starter / AI-agent-assisted development tool
Project type	Reusable design system + code generation framework
What we built	A copy-ready Next.js shell template (Scan/Investigate/Decide/Execute modes, resizable panels, Zustand state, Recharts visualizations) + a

Field	Recommended text
	meta-prompting framework for AI agents to generate new intelligence products from specs
Client/user problem	Internal team needed a way to quickly spin up new operational intelligence dashboards without rebuilding shell chrome, panel layouts, and mode architecture; also wanted a repeatable, spec-driven process for AI-assisted prototype generation with built-in quality gates (Architect/Critic agents)
Main features	Terminal-aesthetic workbench with four operational modes; resizable multi-panel layout; entity-centric navigation; evidence linking; scenario builder with constraints; workflow execution and task management; keyboard shortcuts and command palette; design system tokens and primitives; AI agent prompting pipeline with pre/post-generation audit
Technical stack	Next.js 16, React 19, TypeScript 5, Tailwind CSS v4, Radix UI, Zustand, Recharts, react-resizable-panels, Lucide icons
Proof available	Source code (TypeScript + React); runnable shell with demo data; documentation in `/docs/` and `/prompt/`; AI agent framework with spec templates and checklists
Best sales angle	"A proven shell architecture extracted from a live intelligence platform, delivered as a template. Includes a spec-driven AI agent framework so new products inherit the same UI DNA and operational discipline without design drift. Runnable in minutes; customize only the demo data and modes."
What should not be claimed	Real data ingestion, live market feeds, operational control capabilities, production-ready workflows, automatic relationship detection, advanced analytics, real inference, or that any of the demo metrics reflect actual system state

12. Safe public claims

Based on repository evidence, these claims are safe:

- **"This is a copy-ready Next.js starter template with a terminal-like interface for operational intelligence dashboards."** ✓ Confirmed by README, code structure, and runnable shell.
- **"It implements four canonical operational modes: Scan (watch), Investigate (explore), Decide (analyze), Execute (act)."** ✓ All four modes are fully implemented and functional.
- **"The shell includes resizable multi-panel workbench, keyboard shortcuts, and entity-centric navigation."** ✓ All present in components and app-shell.

- **"It uses React 19, Next.js 16, TypeScript, Tailwind CSS v4, and Zustand for state management."** ✓ Confirmed in package.json.
- **"Includes demo data (mock market indices, assets, evidence, workflows, tasks) intended to be replaced by product-specific content."** ✓ Explicitly documented in README and code.
- **"Bundles a nested design system with reusable primitives, charts, tokens, and layouts."** ✓ `atlas-design-system/` is present and documented.
- **"Provides an AI agent framework for spec-driven prototype generation: Architect (validate), Builder (code), Critic (review)."** ✓ Entire `prompt/` folder with agents, templates, checklists, and HOW_TO_USE guide.
- **"The starter enforces design discipline: compact density, border-led panels, mono metrics, terminal aesthetic, restrained accent usage."** ✓ Documented in `docs/ai-agent-guide.md`, `app/globals.css` (referenced), and visible in components.
- **"Includes pre-generation and post-generation audit checklists and a prompt compiler."** ✓ Both present in `prompt/checklists/` and `prompt/compiler/`.
- **"Built for teams generating multiple intelligence products from a shared template; AI-agent-ready."** ✓ Explicit goal stated in README and docs/ai-agent-guide.md.

13. Unsafe or unproven claims

Do NOT claim publicly without external proof:

- **"This system powers real operational intelligence at Atlas."** ✗ This is a starter template; the README calls it "extracted from the live Atlas terminal," but this repo is the template, not the live system.
- **"Real-time market data and insights are available."** ✗ All data is mock/demo; no actual market feeds.
- **"The Scan mode monitors live market indices and provides real alerts."** ✗ All indices, prices, and alerts are hardcoded demo data.
- **"The Investigate mode analyzes real entity relationships and performs network analysis."** ✗ Entity graph is static coordinate-based; no real relationship engine.

- **"The Decide mode runs scenario modeling and produces recommendations."** ✗ Options and recommendations are static; no computational backend.
- **"The Execute mode manages real operational workflows and tasks."** ✗ Workflows and tasks are mock; clicking buttons does not update state beyond in-memory Zustand store or persist to any backend.
- **"System metrics (latency: 31ms, ingest: 2s ago) reflect real performance."** ✗ Hardcoded strings in bottom-strip.
- **"Evidence reliability and confidence scores are scientifically calculated."** ✗ All scores are hardcoded demo values; no visible algorithm.
- **"The AI agent framework is production-ready and has been tested at scale."** ✗ The framework is operational and documented, but described as experimental/evolving ("direction: AI Product Development OS").
- **"Users can search and navigate real evidence or entity networks."** ✗ Command bar and command palette filter only against static demo entities and evidence.
- **"This product is currently live and in use by customers."** ✗ This is a starter template for internal team and AI agent use, not a customer-facing product.

14. Missing evidence needed

To strengthen a public portfolio case study:

- **Screenshots** of the Scan, Investigate, Decide, Execute modes in action (for website/case study).
- **Demo video** showing keyboard navigation, mode switching, panel resizing, entity selection, context panel filtering.
- **Usage metrics** (if live internally) — e.g., number of products generated from this template, time saved per new product.
- **Testimonial** from internal users or AI agents who have used the starter successfully.
- **Production deployment confirmation** — Is this actually running in Atlas Intelligence? What scale? What usage?
- **Real data source confirmation** — If used in production, where does real data come from? What APIs/databases? How is data loaded?

- **Case study notes** on why this architectural approach (terminal aesthetic, four modes, resizable panels) was chosen and how it improves user experience vs. traditional dashboards.
- **Performance benchmarks** — Build time, bundle size, rendering performance with large datasets.
- **Comparison to competitor starters** or design systems — what makes this unique?
- **Client/user validation** — Did any derived products achieve specific business outcomes?

15. Final confidence score

Confidence: 62/100
Rationale:

1. ✓ **High confidence in technical accuracy:** The codebase is complete, well-structured, and functional. All documented features (modes, resizable panels, state management, charts, keyboard shortcuts) are present and working. No vaporware or incomplete code detected. Code quality is professional.
2. ⚠️ **Medium-low confidence for public portfolio case:** The project is a **template and framework, not a shipping product**. All data is demo/mock. There is no proof of real-world usage, real data integration, or production deployment. The README and docs are honest about this (explicitly labeling demo-data), but marketing this as a public portfolio case requires external proof (screenshots, testimonials, deployment confirmation, real usage metrics). Without that, any claim beyond "we built a well-architected starter template" risks overclaiming.
3. ⚠️ **Medium confidence in AI agent framework viability:** The `prompt/` folder is thoughtfully designed with Architect/Builder/Critic agents and spec templates, but there is no evidence of real-world agent usage, success rates, or iteration data. The framework reads experimental and promising, but not yet battle-tested at the portfolio level.

What would increase confidence to 80+:

- Public testimonial or case study from a derived product (e.g., "We used this template to generate the X dashboard in 2 weeks vs. 6 weeks with traditional approach").

- Proof of production deployment and real usage.
- Performance/bundle size metrics demonstrating advantages.
- Screenshots/demo showing the UI in action.
- Evidence that the AI agent pipeline has successfully generated 5+ products.

What prevents higher confidence:

- Zero production or customer-facing claims; repo is internal-only.
- No metrics on real usage, adoption, or business impact.
- All data is mock; no integration evidence.
- AI agent framework is promising but lacks real-world validation.